



Introduction To Programming Using C++ Level 2

Common Mistakes with Pointers

The background of the slide is a photograph of a lecture hall. In the foreground, the silhouettes of several students are visible, some sitting and some standing, looking towards the front. A large, dark rectangular screen or whiteboard is positioned in the center of the room, serving as the backdrop for the text. The overall lighting is dim, with the screen area being the brightest part of the image.

Introduction To Programming Using C++

Level 2

Common Mistakes with Pointers

Definition

Address and Value

-  When working with pointers, it is easy to confuse an address with a value.
-  We must always pay attention to what a pointer expects: an address or a value.
-  The pointer itself stores an address, while dereferencing the pointer gives us the value stored at that address.

Explanation

Pointer and Address

 If p is a pointer to x , then p stores the address of x .

 Therefore:

$p = \&x;$

Pointer and Value

 If p points to x , then $*p$ accesses the value stored in x .

 Therefore:

$*p = x;$

Address of a Variable

 The $\&$ operator is used to get the address of a variable.

 Therefore:

$\&x$

Procedures

1. Create a normal variable.
2. Create a pointer of the same type.
3. Assign the address of the variable to the pointer.
4. Remember that the pointer stores an address.
5. Use `*p` to access the value stored at that address.
6. Use `&x` to get the address of `x`.
7. Do not confuse `p` with `*p` or `x` with `&x`.

💡 Example

◆ Correct and Incorrect Pointer Usage

```
int x = 10;
```

```
int* p;
```

```
p = &x;    // ✓ Correct
```

```
cout << p << endl;    // Address of x
```

```
cout << &x << endl;    // Address of x
```

```
cout << *p << endl;    // Value of x
```

```
cout << x << endl;    // Value of x
```

Note

◆ A simple way to remember the relationship is:

$x \rightarrow \text{Value}$

$\&x \rightarrow \text{Address of } x$

$p \rightarrow \text{Address stored in the pointer}$

$*p \rightarrow \text{Value at the address stored in } p$

⊘ Warning

- ◆ Do not confuse `p` with `*p`.
- ◆ `p` represents the address stored in the pointer.
- ◆ `*p` represents the value stored at that address.
- ◆ Do not confuse `x` with `&x`.
- ◆ `x` represents the value of the variable.
- ◆ `&x` represents the address of the variable.
- ◆ Also remember that `*` has different uses depending on the context:
 - ▣ `int* p;`
→ Declares `p` as a pointer.
 - ▣ `*p`
→ Dereferences `p` and accesses the value.



Comparison



p

- ✓ Stores the address of x.
- ✓ Example:
 $p = \&x;$



*p

- ✓ Accesses the value stored at the address in p.
- ✓ Example:
 $*p = x;$



&x

- ✓ Gives the address of x.
- ✓ Example:
 $p = \&x;$



x

- ✓ Represents the value stored in x.
- ✓ Example:
 $*p = x;$

✦ Summary

- ✓ $p \rightarrow$ Address stored in the pointer.
- ✓ $*p \rightarrow$ Value at the address stored in the pointer.
- ✓ $x \rightarrow$ Value stored in variable x .
- ✓ $\&x \rightarrow$ Address of variable x .
- ✓ $p = \&x \rightarrow$ Stores the address of x in p .
- ✓ $*p = x \rightarrow$ Accesses the same value stored in x .
- ✓ Use $\&$ when you need the address.
- ✓ Use $*$ when you need to dereference a pointer and access the value.

Thank you 😊